DESENVOLVIMENTO MOBILE · MÓDULO 20 · TRILHA FLUTTER · NÍVEL AVANÇADO

Tópicos Avançados e Projeto Final

IA no aparelho e na nuvem, segurança, monetização, outras superfícies, carreira e o projeto integrador

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar treinamento, inferência e o funcionamento dos modelos de linguagem.
- Usar IA no app Flutter: ML Kit, modelos próprios com LiteRT e Gemini via Firebase.
- Analisar ameaças com STRIDE e revisar o app com o OWASP MASVS.
- Escolher e implementar um modelo de monetização.
- Planejar, construir e apresentar o projeto final do curso.

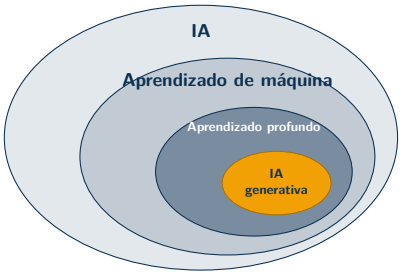
Roteiro

Neste módulo

- Fundamentos de inteligência artificial
- Fundamentos de segurança da informação
- Fundamentos de produto e negócio
- IA no app Flutter
- Segurança no app Flutter
- Monetização e outras superfícies
- Carreira e portfólio
- Projeto final
- Fechamento do curso

Fundamentos de inteligência artificial

IA, aprendizado de máquina e aprendizado profundo



Aprendizado de máquina (ML)

Em vez de programar as regras, o sistema **aprende** padrões a partir de exemplos (dados) e os aplica a casos novos.

Paradigma	Aprende com
Supervisionado	exemplos rotulados (leitura → "normal" ou "falha")
Não supervisionado	dados sem rótulo, buscando grupos e anomalias
Por reforço	tentativa e erro, guiado por recompensas

Aprendizado profundo

ML com redes neurais de muitas camadas; base da visão computacional, da fala e dos modelos de linguagem.

Treinamento e inferência



Treinamento

Ajustar milhões de parâmetros (**pesos**) para que o modelo erre o mínimo nos exemplos. Caro e feito uma vez (ou de tempos em tempos).

Inferência

Usar o modelo pronto para responder a uma entrada nova. É o que o app faz, milhares de vezes.

Quantização

Trocar pesos de 32 bits por inteiros de 8 bits: modelo cerca de 4 vezes menor e mais rápido, com pequena perda de precisão.

Avaliar antes de confiar

Em detecção de falhas, **precisão** (dos alertas, quantos eram reais) e **revocação** (das falhas reais, quantas foram alertadas) importam mais que a acurácia, porque falhas são raras.

Modelos de linguagem (LLMs)

LLM — *Large Language Model*

Rede neural treinada com enormes volumes de texto para prever o próximo ***token*** (pedaço de palavra). Gerando um *token* após o outro, escreve respostas, resume, traduz e segue instruções.

Termo	Significado
Janela de contexto	quanto texto o modelo considera de uma vez
Temperatura	grau de aleatoriedade da resposta
Alucinação	resposta fluente, mas falsa
RAG	buscar documentos relevantes e incluí-los na pergunta
Chamada de ferramentas	o modelo pede que o app execute uma função

Analogia

O corretor do teclado levado ao extremo: em vez da próxima palavra, continua parágrafos inteiros.

Atenção

O modelo não "sabe" se está certo: produz o texto mais provável. A resposta é uma **sugestão** que uma pessoa confere.

Dica

Com RAG, o LabMonitor responde com base nos manuais dos equipamentos, citando a fonte.

Engenharia de *prompt* e uso responsável

Estrutura de um bom *prompt*

```
Papel:     Você é assistente técnico de um
           laboratório de eletroeletrônica.
Contexto:  Leituras da bancada B3 (última hora):
           {leituras}
Tarefa:    Explique o alerta de temperatura.
Formato:   No máximo 3 frases, em português,
           e termine com uma ação recomendada.
Limites:   Se os dados não bastarem, diga isso.
```

Dica

Dê exemplos de entrada e saída esperadas (*few-shot*) e peça respostas em JSON quando o app precisar processá-las.

Injeção de *prompt*

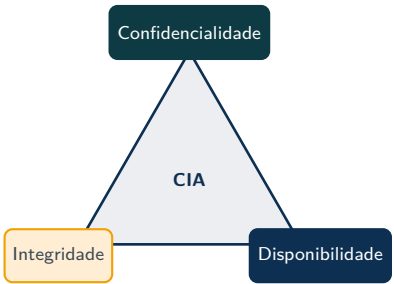
Texto vindo do usuário ou de documentos pode conter instruções ("ignore as regras anteriores..."). Nunca dê ao modelo poder de executar ações sensíveis sem confirmação e validação no servidor.

Uso responsável

- Informe que a resposta foi gerada por IA.
- Minimize dados pessoais enviados (LGPD).
- Limite custos: cada *token* é cobrado.
- Avalie com casos de teste, como qualquer código.

Fundamentos de segurança da informação

Princípios de segurança



Tríade CID (em inglês, CIA).

Propriedade	Pergunta
Confidencialidade	só quem pode vê os dados?
Integridade	os dados não foram alterados indevidamente?
Disponibilidade	o serviço funciona quando precisam dele?
Autenticidade	quem diz ser é quem é?

Princípios de projeto

Menor privilégio (cada parte com o mínimo de acesso), **defesa em profundidade** (várias camadas), **seguro por padrão** e **nunca confiar no cliente**: o app pode ser modificado por quem o instala.

Criptografia essencial

	Hash	Simétrica	Assimétrica
Ideia	resumo de tamanho fixo, impossível de reverter	a mesma chave cifra e decifra	par de chaves: pública e privada
Exemplos	SHA-256; para senhas, Argon2 ou bcrypt	AES-GCM	RSA, curvas elípticas (ECDSA, X25519)
Uso no app	integridade de arquivos, armazenar senhas no servidor	dados locais cifrados, conteúdo do canal TLS	assinatura do app, troca de chaves, *passkeys*

Como o HTTPS junta tudo

O TLS usa criptografia assimétrica para autenticar o servidor (certificado) e combinar uma chave secreta; depois cifra o tráfego com criptografia simétrica, que é muito mais rápida.

Criptografia caseira

Inventar algoritmos, reutilizar vetores de inicialização ou guardar a chave junto com o dado cifrado. Use bibliotecas e o cofre de chaves do sistema (Keystore, Keychain).

Modelagem de ameaças com STRIDE

Ameaça	Exemplo no LabMonitor	Defesa
Spoofing (falsificação)	alguém se passa por um ESP32 e envia leituras falsas	autenticar dispositivos (certificados, MQTT com senha)
Tampering (adulteração)	alterar a reserva no tráfego	TLS, validação no servidor
Repudiation (repúdio)	aluno nega ter cancelado a reserva	registros de auditoria
Information disclosure	token salvo em texto puro	armazenamento seguro
Denial of service	app em laço enviando milhares de pedidos	limites de taxa no servidor
Elevation of privilege	aluno vira administrador alterando o app	regras e papéis no servidor

Modelagem de ameaças

Analisar sistematicamente o que pode dar errado, **antes** de construir: desenhar o fluxo de dados, listar ameaças, priorizar e definir defesas.

Um pouco de história

STRIDE foi criado na Microsoft no fim dos anos 1990 e virou uma das técnicas mais ensinadas.

OWASP Mobile Top 10 e MASVS

OWASP Mobile Top 10 (2024)	
M1	uso inadequado de credenciais
M2	segurança insuficiente da cadeia de suprimentos
M3	autenticação e autorização inseguras
M4	validação insuficiente de entradas e saídas
M5	comunicação insegura
M6	controles de privacidade inadequados
M7	proteção insuficiente do binário
M8	configuração de segurança incorreta
M9	armazenamento de dados inseguro
M10	criptografia insuficiente

OWASP

Fundação aberta que publica guias de segurança. O **Top 10** lista os riscos mais comuns; o **MASVS** define os requisitos de segurança para apps, e o **MASTG** ensina a testá-los.

Grupos do MASVS

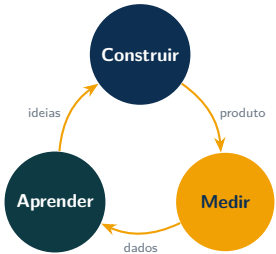
Armazenamento, criptografia, autenticação, rede, plataforma, código, resiliência e privacidade.

Dica

Use o MASVS como *checklist* de revisão antes de cada versão importante.

Fundamentos de produto e negócio

Do problema ao MVP



MVP — Produto Mínimo Viável

A menor versão do produto que permite **aprender** com usuários reais se a solução resolve o problema. Não é um produto mal feito: é um produto pequeno.

Métricas AARRR

Aquisição (chegam), **A**tivação (têm uma boa primeira experiência), **R**etenção (voltam), **R**eceita (pagam) e **R**ecomendação (indicam).

Um pouco de história

O ciclo construir–medir–aprender foi popularizado por Eric Ries no livro *A Startup Enxuta* (2011).

Modelos de receita e métricas do negócio

Modelo	Como gera receita
Pago	paga-se para instalar
Freemium	grátis, com recursos pagos
Assinatura	cobrança recorrente
Anúncios	exibições e cliques
B2B / institucional	a organização paga pelo serviço
Público / acadêmico	financiado por projeto ou órgão

Métrica	Definição
ARPU	receita média por usuário ativo no período
Churn	% de assinantes que cancelam por mês
LTV	receita esperada de um usuário durante toda a relação; aproximação: ARPU mensal $\times$ margem $\div$ *churn* mensal
CAC	custo para adquirir um usuário (marketing $\div$ novos usuários)

Conta rápida

ARPU de R$ 10/mês, margem de 70 % e *churn* de 5 %/mês: LTV $\approx 10 \times 0{,}7 \div 0{,}05 =$ R$ 140. Gastar R$ 200 para adquirir cada usuário dá prejuízo.

IA no app Flutter

Onde roda o modelo?

	No aparelho	Na nuvem
Latência	muito baixa	depende da rede
Offline	sim	não
Privacidade	dados ficam no aparelho	dados saem do aparelho
Custo	nenhum por uso	por requisição (*tokens*)
Modelo	pequeno (KB a centenas de MB)	muito grande

Pacote	O que oferece
`google_mlkit_*`	modelos prontos: texto (OCR), códigos, rostos, rótulos, tradução
`tflite_flutter`	roda o **seu** modelo `.tflite` (LiteRT)
`firebase_ai`	modelos Gemini na nuvem, sem expor chaves

LiteRT

Novo nome do **TensorFlow Lite**: executa modelos treinados em Python, com aceleração por GPU ou NPU quando disponível.

Dica

Comece pelo mais simples que resolve: regra estatística, depois modelo pronto, depois modelo próprio, e só então um LLM.

Anomalias nas leituras e OCR

1) Linha de base: z-score em uma janela

```dart
Stream<double> anomalias(Stream<double> leituras,
    {int janela = 60, double limiar = 3}) async* {
  final buf = ListQueue<double>();
  await for (final x in leituras) {
    if (buf.length == janela) {
      final m = buf.reduce((a, b) => a + b) / janela;
      final v = buf.map((y) => (y - m) * (y - m))
          .reduce((a, b) => a + b) / janela;
      final dp = sqrt(v);
      if (dp > 0 && (x - m).abs() / dp > limiar) yield x;
      buf.removeFirst();
    }
    buf.addLast(x);
  }
}
```

z-score

Distância da leitura à média recente, em desvios-padrão. Acima de 3, é muito improvável.

2) Modelo próprio (tflite_flutter)

```dart
final it = await Interpreter.fromAsset(
    'assets/anomalia.tflite');
final saida = [List.filled(1, 0.0)];   // [1, 1]
it.run([janelaNormalizada], saida);    // [1, 60]
```

3) OCR da placa do equipamento

```dart
final ocr = TextRecognizer(
    script: TextRecognitionScript.latin);
final r = await ocr.processImage(
    InputImage.fromFilePath(foto.path));
final patrimonio = RegExp(r'\d{6}')
    .firstMatch(r.text)?.group(0);
await ocr.close();
```

Dica

Compare qualquer modelo com a linha de base: às vezes ela basta.

Gemini pelo Firebase AI Logic

data/services/assistente_service.dart

```dart
import 'package:firebase_ai/firebase_ai.dart';

final _modelo = FirebaseAI.googleAI().generativeModel(
  model: nomeDoModelo,        // um modelo Gemini atual
  systemInstruction: Content.system(
    'Você é assistente técnico de um laboratório de '
    'eletroeletrônica. Responda em até 3 frases e diga '
    'quando os dados não forem suficientes.'),
);

Future<String> explicarAlerta(String leituras) async {
  final r = await _modelo.generateContent([
    Content.text('Leituras da bancada B3: $leituras. '
        'Por que o alerta de temperatura disparou?'),
  ]);
  return r.text ?? 'Sem resposta do assistente.';
}
```

Por que pelo Firebase

O app não guarda chave de API: o Firebase intermedeia a chamada, e o **App Check** (Play Integrity no Android, App Attest no iOS) garante que só o seu app consome a cota.

Atenção

Mostre a resposta como **sugestão**, identificada como gerada por IA, e trate falhas e limites de uso. Modelos e preços mudam: consulte a lista atual antes de escolher.

Segurança no app Flutter

Revisão MASVS de um app Flutter

Grupo	O que fazer no Flutter
Armazenamento	segredos em `flutter_secure_storage`; nada sensível em `shared_preferences`, logs ou capturas
Criptografia	bibliotecas padrão (`cryptography`, cofres do sistema); nunca algoritmos próprios
Autenticação	tokens curtos e renováveis; biometria só para desbloquear segredos locais
Rede	HTTPS sempre; *pinning* em apps sensíveis, com plano de troca do certificado
Plataforma	validar *deep links* e dados de QR codes; WebView sem JavaScript desnecessário
Código	`-obfuscate`; nenhum segredo no binário; dependências auditadas (`dart pub outdated`)
Resiliência	Firebase App Check ou verificação de integridade no servidor
Privacidade	coletar o mínimo; declarações das lojas coerentes com o código

O cliente nunca é confiável: toda regra importante é verificada também no servidor.

Pinning de certificado

Só aceitar o certificado da API do LabMonitor

```dart
import 'dart:io';
import 'package:flutter/services.dart';
import 'package:http/io_client.dart';

Future<IOClient> clienteComPinning() async {
  final pem = await rootBundle.load('assets/certs/api.pem');
  final ctx = SecurityContext(withTrustedRoots: false)
    ..setTrustedCertificatesBytes(pem.buffer.asUint8List());
  return IOClient(HttpClient(context: ctx));
}
```

Certificate pinning

O app confia apenas em um certificado (ou chave) específico, e não em qualquer autoridade certificadora instalada no aparelho. Dificulta interceptar o tráfego com um certificado falso.

Atenção

Certificados expiram. Fixe a chave da autoridade intermediária ou mantenha dois certificados válidos, e planeje a troca antes do vencimento, ou o app para de funcionar.

Monetização e outras superfícies

Compras no app

in_app_purchase (trechos)

```
final iap = InAppPurchase.instance;
iap.purchaseStream.listen((compras) async {
  for (final c in compras) {
    if (c.status == PurchaseStatus.purchased) {
      final ok = await api.validarCompra(
          c.verificationData.serverVerificationData);
      if (ok) liberarRecursoPremium();
    }
    if (c.pendingCompletePurchase) {
      await iap.completePurchase(c);
    }
  }
});

final r = await iap.queryProductDetails({'relatorios_pro'});
await iap.buyNonConsumable(purchaseParam:
    PurchaseParam(productDetails: r.productDetails.first));
```

Regras das lojas

Bens digitais usados no app passam pelo sistema de compras da loja, que retém uma comissão; bens físicos seguem outras regras.

Atenção

Valide cada compra **no servidor** com as APIs da Play e da App Store antes de liberar o conteúdo.

Dica

Anúncios: `google_mobile_ads`. Serviços como RevenueCat simplificam assinaturas nas duas lojas.

Além da tela do celular



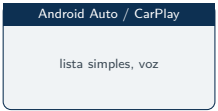
Widgets da tela inicial

O pacote `home_widget` envia dados do Flutter; o visual é nativo (Glance no Android, WidgetKit no iOS).



Relógios

Wear OS e Apple Watch costumam exigir apps ou extensões nativas; o app Flutter no celular envia os dados.



Carro e TV

Usam *templates* e regras de segurança próprias de cada sistema, com integração nativa.

Dica

A arquitetura em camadas permite reaproveitar repositórios e serviços em todas essas superfícies; só a apresentação muda.

Carreira e portfólio

Do curso ao mercado

Portfólio que convence

- Dois ou três apps completos no GitHub, com README, capturas e arquitetura explicada.
- Pelo menos um app publicado (mesmo em teste aberto).
- Testes e CI visíveis no repositório.
- Um projeto com hardware (BLE, IoT) é um diferencial raro.

Continue aprendendo

- Documentação oficial, *Flutter samples* e o canal do Flutter no YouTube.
- Notas de versão do Flutter e do Dart a cada lançamento.
- Comunidades: GDG, Flutter Brasil, eventos locais.
- Contribuir com pacotes de código aberto no pub.dev.

Assistentes de IA

Use-os para acelerar, mas entenda e revise cada linha: você responde pelo código que publica.

Projeto final

Projeto final: requisitos

Requisitos mínimos

1. Quatro ou mais telas com `go_router` e Material 3.
2. MVVM com repositórios, serviços e injeção de dependência.
3. Banco local (Drift) e preferências.
4. API REST própria ou BaaS (Módulos 16 e 17).
5. Funcionamento offline nas telas principais.
6. Um recurso do dispositivo (BLE, câmera, GPS...).
7. Testes de unidade e de widget; CI no GitHub.
8. Acessibilidade, tema escuro e build de release.

Sugestões de tema

- Concluir e publicar o **LabMonitor**.
- Empréstimo de equipamentos do laboratório com QR code.
- Medidor de consumo de energia com ESP32 e sensor de corrente.
- Acompanhamento de estágio ou TCC.

Dica

Resolva um problema real de alguém que você conhece e valide o MVP com essa pessoa.

Entregáveis, cronograma e avaliação

Semana	Marco
1	proposta: problema, público, protótipo no Figma, arquitetura e modelo de ameaças
2	navegação, UI e dados locais
3	backend, recurso do dispositivo e offline
4	testes, acessibilidade, release e apresentação

Entregáveis

Repositório no GitHub com README, build de release (AAB ou link de teste), vídeo de 3 minutos e apresentação de 10 minutos.

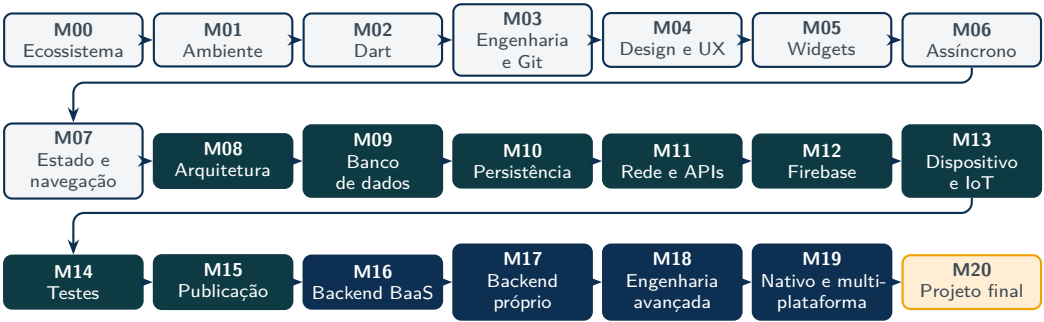
Critério de avaliação	Peso
Funcionalidades e requisitos atendidos	25 %
Arquitetura e qualidade do código	20 %
Experiência do usuário, UI e acessibilidade	15 %
Testes automatizados e CI	15 %
Recursos do dispositivo e integração	10 %
Documentação (README, diagramas)	5 %
Apresentação e demonstração	10 %
Total	**100 %**

Dica

Pesos sugeridos: ajuste ao plano de ensino.

Fechamento do curso

A jornada completa



O que você construiu

Um app Flutter completo e publicado, com arquitetura profissional, dados locais e remotos, backend, integração com hardware, testes e entrega contínua, rodando no celular, na web e no desktop.

Glossário

Termo	Significado
Aprendizado de máquina	técnica em que o sistema aprende padrões a partir de dados
Inferência	uso de um modelo treinado para responder a entradas novas
Quantização	redução da precisão dos pesos (ex.: 32 para 8 bits) para encolher o modelo
LLM	modelo de linguagem que gera texto prevendo o próximo *token*
RAG	técnica que inclui documentos recuperados na pergunta ao LLM
STRIDE	classificação de ameaças: falsificação, adulteração, repúdio, vazamento, negação de serviço, elevação de privilégio
MASVS	padrão da OWASP com os requisitos de segurança de apps móveis
MVP	menor versão do produto que permite aprender com usuários reais
LTV / CAC	valor de um cliente ao longo da relação / custo para adquiri-lo

Questões de revisão

1. Diferencie treinamento de inferência. Onde cada um costuma acontecer num app?
2. Por que precisão e revocação são mais úteis que a acurácia para avaliar um detector de falhas?
3. O que é alucinação em LLMs e como o app deve lidar com ela?
4. Por que a chave de API de um LLM não pode ficar dentro do app? Qual a alternativa?
5. Aplique STRIDE a uma funcionalidade do seu projeto final e proponha uma defesa por ameaça.
6. Por que a compra no app deve ser validada no servidor?
7. Qual o risco do *certificate pinning* e como mitigá-lo?
8. Calcule o LTV com ARPU de R$ 8/mês, margem de 60 % e *churn* de 4 %/mês.

Exercícios finais

Exercício 1 — IA

Gere 24 h de leituras simuladas com picos artificiais e compare o detector por z-score com um modelo treinado em Python e convertido para LiteRT: medir precisão e revocação de cada um.

Exercício 2 — Segurança

Revise o LabMonitor com o *checklist* MASVS e o STRIDE e registre ao menos três correções como *issues* no GitHub.

Exercício 3 — Assistente

Implemente "explique este alerta" com o Firebase AI Logic, marcando a resposta como gerada por IA.

Projeto final

Escolha o tema, entregue a proposta na semana 1 e siga o cronograma de quatro semanas.

Para aprofundar

- ML Kit para Flutter: `https://pub.dev/packages/google_ml_kit`
- LiteRT: `https://ai.google.dev/edge/litert`
- Firebase AI Logic: `https://firebase.google.com/docs/ai-logic`
- OWASP MASVS e MASTG: `https://mas.owasp.org`
- Compras no app: `https://pub.dev/packages/in_app_purchase`
- Segurança no Flutter: `https://docs.flutter.dev/security`

Respostas das questões de revisão

1. Treinamento ajusta os pesos com muitos dados (servidor com GPU); inferência usa o modelo pronto para uma entrada nova (no aparelho ou na nuvem, a cada uso).
2. Falhas são raras: um detector que nunca alerta acerta quase sempre, mas é inútil. Precisão mede alarmes falsos; revocação mede falhas não detectadas.
3. É uma resposta fluente e falsa. O app deve apresentá-la como sugestão, citar fontes quando houver (RAG) e pedir confirmação humana para decisões.
4. O binário pode ser extraído e a chave, usada por terceiros às suas custas. A chamada deve passar por um backend ou serviço intermediário (Firebase AI Logic com App Check).
5. Exemplo para a reserva: falsificação de usuário (autenticação), adulteração da reserva (TLS e validação no servidor), elevação de privilégio (papéis verificados no servidor).
6. O app pode ser modificado para fingir a compra; só o servidor, consultando a loja, confirma que o pagamento existe.
7. Se o certificado mudar, o app para de se conectar. Mitigação: fixar a chave da autoridade intermediária, manter um certificado reserva e planejar a troca.
8. LTV $\approx 8 \times 0{,}6 \div 0{,}04 = \text{R\$}\,120$.

Obrigado!

Prof. Dr. Francisco dos Santos Viana

Fim do curso Desenvolvimento Mobile — trilha Flutter